

README

Helix OTA — Future Phase: Other Operating Systems (macOS, *BSD, RTOS, and upstream OSes)

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	future (research outline)
Status summary	Basic planning for extending Helix OTA beyond Android/Linux/Windows to all other upstream operating systems via the OS-adapter seam: macOS, *BSD, and constrained/RTOS targets. The control plane is OS-agnostic; only device adapters are added.
Issues	Very heterogeneous: from full desktop OSes with their own update frameworks to microcontroller RTOS with bespoke bootloaders. The adapter abstraction must stretch across all.
Issues summary	The OS-adapter contract is the universality guarantee; each new OS is “just” a new adapter + a spike.
Fixed	initial future outline
Fixed summary	Captured the long-tail OS targets and how they map onto the universal seam.
Continuation	Promote individual OSes to numbered phases (e.g. 4.0.0-universal-orchestrator) as demand arises; each gets full specs + a spike.

Table of contents

- [§1. Scope & principle](#)

- §2. Candidate targets & mechanisms
- §3. The plugin/adaptor registry
- §4. Open questions

§1. Scope & principle

The operator mandate is ultimate, universal coverage. The architectural guarantee that makes this tractable is the **OS-adaptor seam** plus the OS-agnostic control plane, rollout engine, trust layer, and telemetry schema. Adding an OS never touches the server — it adds a device adaptor implementing `ota-protocol` + the adaptor interface, plus a spike.

§2. Candidate targets & mechanisms

- **macOS** — signed/notarized `.pkg` + installer, MDM commands (`InstallApplication/ScheduleOSUpdate`), Sparkle for app-level auto-update; SecureBoot/SIP interplay.
- ***BSD (FreeBSD/OpenBSD/NetBSD)** — `freebsd-update/pkg`, ZFS boot-environment A/B (beadm-style) for atomic rollback — a strong fit for the safe-update model.
- **RTOS / MCU (Zephyr, FreeRTOS, Mbed)** — **MCUboot** signed A/B image swap is the dominant standard; SUIT (RFC 9019/9124) manifest model; relevant if Helix ever services downstream micro-devices (maps to the Uptane secondary-ECU tier deferred in 1.0.1).
- **ChromeOS / other immutable OSes** — A/B partition updates (same conceptual model as Android).

§3. The plugin/adaptor registry

A registry of OS adaptors (master design §4; `research/stacks/commercial-oss-fleet.md` for the open-client/closed-backend lesson Helix inverts). Each adaptor declares capabilities (`GetCapabilities`) so the control plane can target by capability, not by hard-coded OS. This is also where an SUIT/Uptane secondary-ECU tier would attach for micro-device fan-out.

§4. Open questions

Which OSes have a reliable post-install health signal (for health-gated halt)? Which support true A/B/atomic rollback vs best-effort? Where must Helix supply the bootloader integration vs reuse the platform's? Each becomes a spike before its phase ADR.

§5. addition_3_research_routing (rev 2 — 2026-06-08)

This section folds addition-#3's universal/multi-OS design into this canonical phase dir per synthesis §11. It **extends** the original outline above; nothing above is removed. The source doc self-numbered `2.0.0-multi-os-universal`; synthesis §11 routes it to `1.X-other-os/ (or 2.0.0-multi-os-universal/)` — captured here as the universal-adapter platform that this `1.X-other-os` dir already describes. The eventual `2.0.0` version number is preserved as the platform's own roadmap target (source §7).

§5.1 source_research

Source: `additions/initial_research_03/helix_ota_big/2.0.0-multi-os-universal/docs/MULTI_OS_UNIVERSAL_DESIGN.md`. Cited sections, summarized (NOT copied):

- **§1 Universal OTA Vision** — single-platform thesis (§1.1), core design principles (§1.2), the **plugin architecture as the universality abstraction** (§1.3), cross-OS dependency management (§1.4) — the same OS-adapter-seam principle stated in §1 above.
- **§2 Plugin Architecture** — dual-mode plugin runtime (§2.1), the **full OSAdapter interface definition** (§2.2), adapter lifecycle state machine (§2.3), adapter manager (§2.4), an adapter marketplace/registry (§2.5) — the concrete form of the §3 adapter registry above — and a third-party adapter SDK (§2.6).
- **§3 Supported OS Catalog** — mobile / desktop / server / embedded / IoT / other (§3.1–§3.6), aligning with the candidate targets in §2 above (macOS, *BSD, RTOS/MCU, ChromeOS).
- **§4 OS-Agnostic Update Pipeline** — universal update lifecycle (§4.1), adapter-to-pipeline mapping (§4.2), server-side version compatibility matrix (§4.3), artifact-format abstraction layer (§4.4).
- **§5 Cross-OS Dashboard** — unified fleet view, OS-specific health indicators, cross-OS rollout coordination.
- **§6 New submodules** — `helix-plugin-sdk`, `helix-plugin-registry`, `helix-adapter-freebsd/macos/yocto/rtos`.
- **§7 Roadmap beyond 2.0.0** — version timeline (macOS 2.1.0, embedded/RTOS 2.2.0, iOS 2.3.0 limited by Apple, predictive-rollback 3.0.0) plus appendices (interface reference, plugin-manifest schema, risk matrix, conformance test matrix).

§5.2 reconciliation_to_locked_decisions

- **Control plane unchanged + OS-agnostic:** Go + **Gin**, REST primary; the universal pipeline, trust layer, rollout engine, and telemetry schema are reused — adding an OS adds an adapter, never a server fork. This matches the §1 principle above and the locked OS-adapter seam (master design §4).
- **Vocabulary:** canonical **releases + deployments**; re-base updates/rollouts.
- **Signing/trust:** **ed25519 + SHA-256** as the OS-agnostic trust layer (the TUF/Uptane seam carries over); per-OS native signing (Authenticode, X.509/CMS, notarization, MCUboot/SUIT) layers on top.
- **Adapter SDK / marketplace:** accepted as the registry described in §3 above; **defer** the third-party-marketplace ambitions until at least three first-party

adapters (Linux/Windows + one other) are proven. The dual-mode plugin runtime (in-process vs out-of-process) needs an ADR — a plugin-loading model is a security-sensitive choice.

- **Module path / submodules:** re-base helix-* to ota-* under `github.com/HelixDevelopment`; PUBLIC repo creation gated on G11.
- **Coverage:** $\geq 90\%$ floor on the adapter-conformance and trust-layer paths.

§5.3 what_must_be_specced_before_this_phase_starts

1. **Freeze the canonical OSAdapter interface** as the single seam shared by Android/Linux/Windows/other — this is the gating artifact for the whole universality program; it must be specced before any second-OS adapter (i.e. before 1.X-linux/windows promotion).
2. ADR on the **plugin runtime model** (in-process vs sandboxed out-of-process) — security and stability sensitive.
3. Artifact-format abstraction + server-side cross-OS compatibility-matrix spec.
4. Cross-OS dashboard spec (ties to the dashboard gap G6).
5. Per-OS adapter specs + spikes (FreeBSD ZFS boot-env, macOS pkg/MDM, Yocto, RTOS MCUboot/SUIT) — each with a reliable post-install health signal confirmed.
6. Conformance test matrix (source Appendix D) re-based to the four-layer strategy.

§5.4 anti_bluff_unverified_register

Per Constitution §11.4.6 / §7.1 — MUST NOT propagate as fact:

- The entire **roadmap timeline** (2.1.0/2.2.0/2.3.0/3.0.0 dates and the “AI-powered predictive rollback” item, source §7) — aspirational, UNVERIFIED; not a commitment.
- Per-OS adapter feasibility (FreeBSD/macOS/Yocto/RTOS/iOS) — UNVERIFIED until spiked; **iOS is explicitly constrained by Apple** and may be infeasible.
- Plugin-runtime / marketplace claims — UNVERIFIED design; gated on the runtime-model ADR.
- Source-doc Go interface/SDK reference + helix-* submodule names — reference only; non-canonical until re-based.
- Source risk-matrix and conformance assertions — UNVERIFIED until validated.
- All cited HelixConstitution §11.4.x clause numbers remain UNVERIFIED except the six confirmed in `tests/test_strategy.md` §13.